

ÉLÉMENTS DE CORRECTION — 10 SCÉNARIOS D'ATTAQUE WEB

Les réponses ci-dessous représentent les **éléments essentiels attendus**.

Le stagiaire peut utiliser une formulation différente si le raisonnement est correct.

SCÉNARIO 1 — Entrée utilisateur insérée dans une page HTML

Vulnérabilité

XSS (Cross-Site Scripting)

Justification

La valeur saisie par l'utilisateur est insérée directement dans la page HTML sans échappement. Un contenu interprétable par le navigateur peut donc être injecté.

Exploitation / PoC — éléments attendus

- Identifier le paramètre q.
- Envoyer une valeur contenant du contenu HTML/JavaScript de test.
- Le serveur récupère cette valeur.
- La valeur est réinsérée dans la page.
- Le navigateur interprète le contenu.
- Le comportement permet de confirmer la vulnérabilité.

Impact potentiel

- Exécution de JavaScript dans le navigateur.
- Modification du contenu de la page.
- Vol potentiel d'informations accessibles côté navigateur.
- Actions réalisées dans le contexte de l'utilisateur.

Prévention

Techniques :

- Encoder/échapper les sorties HTML.
- Valider les entrées.

Organisationnelles :

- Appliquer des règles de développement sécurisé.
- Effectuer des revues de code.

Tests automatisés :

- Tester automatiquement les champs et paramètres reflétés dans les pages.

SCÉNARIO 2 — Requête SQL construite par concaténation

Vulnérabilité

Injection SQL

Justification

Les valeurs username et password sont directement concaténées dans la requête SQL. Une entrée spécialement construite peut modifier la logique de la requête.

Exploitation / PoC — éléments attendus

- Identifier les champs utilisés dans la requête.
- Envoyer une entrée de test contenant une syntaxe SQL.
- Observer le comportement de l'application.
- La saisie est intégrée directement à la requête.
- La structure ou la condition SQL peut être modifiée.
- Le comportement anormal confirme le risque d'injection.

Impact potentiel

- Contournement possible de l'authentification.
- Lecture ou modification de données.
- Suppression de données selon les privilèges de la base.

Prévention

Techniques :

- Utiliser des requêtes préparées/paramétrées.
- Valider les entrées.

Organisationnelles :

- Former les développeurs au développement sécurisé.
- Réaliser des revues de code.

Tests automatisés :

- Tester les paramètres transmis aux requêtes SQL.

SCÉNARIO 3 — Modification de /invoices/{id}

Vulnérabilité

Broken Access Control / IDOR

Justification

L'application vérifie l'identifiant du document mais ne vérifie pas que la ressource appartient à l'utilisateur connecté.

Exploitation / PoC — éléments attendus

- Se connecter avec un compte utilisateur autorisé.
- Consulter une facture légitime.
- Observer l'identifiant dans l'URL.
- Modifier cet identifiant.
- Envoyer la nouvelle requête.
- Vérifier si la facture d'un autre utilisateur devient accessible.

Impact potentiel

- Accès non autorisé aux documents.
- Divulgence de données confidentielles.
- Modification possible de ressources si d'autres opérations sont vulnérables.

Prévention

Techniques :

- Vérifier l'autorisation côté serveur pour chaque ressource.
- Appliquer le principe du moindre privilège.

Organisationnelles :

- Définir clairement les droits des utilisateurs.
- Tester régulièrement les contrôles d'accès.

Tests automatisés :

- Tester l'accès aux ressources avec plusieurs comptes et rôles.

SCÉNARIO 4 — Requête POST sans protection CSRF

Vulnérabilité

CSRF — Cross-Site Request Forgery

Justification

Une action sensible peut être exécutée sans token anti-CSRF alors que le navigateur envoie automatiquement le cookie de session.

Exploitation / PoC — éléments attendus

- La victime est authentifiée sur l'application.
- Une page externe provoque l'envoi d'une requête vers l'application.

- Le navigateur joint automatiquement le cookie de session.
- Le serveur considère la requête comme provenant de l'utilisateur.
- L'action est exécutée sans vérification supplémentaire.
- Le transfert non souhaité permet de constater le problème.

Impact potentiel

- Actions réalisées à l'insu de l'utilisateur.
- Modification de données.
- Transactions non autorisées.

Prévention

Techniques :

- Utiliser des tokens (Jetons) anti-CSRF.
- Configurer correctement les cookies, notamment SameSite.

Organisationnelles :

- Identifier les opérations sensibles.
- Intégrer les contrôles CSRF aux règles de développement.

Tests automatisés :

- Vérifier que les actions sensibles refusent une requête sans token valide.

SCÉNARIO 5 — Authentification sans limitation

Vulnérabilité

Protection insuffisante contre les attaques automatisées sur l'authentification

Justification

Il n'existe ni « Rate Limiting », ni verrouillage adaptatif, ni MFA, ni surveillance des tentatives.

Exploitation / PoC — éléments attendus

- Identifier le formulaire de connexion.
- Effectuer plusieurs tentatives avec des identifiants incorrects.
- Vérifier que les requêtes continuent d'être acceptées.
- Observer l'absence de délai ou de blocage.
- Vérifier l'absence de mécanisme supplémentaire.
- Constater que les tentatives automatisées sont possibles.

Impact potentiel

- Brute force.
- Credential stuffing.
- Compromission de comptes.

Prévention

Techniques :

- Rate limiting / délai ou verrouillage adaptatif.
- MFA.
- CAPTCHA adaptatif en cas de comportement suspect.

Organisationnelles :

- Politique de mots de passe adaptée.
- Procédure de surveillance des tentatives suspectes.

Tests automatisés :

- Tester la réaction de l'application après plusieurs échecs consécutifs.

SCÉNARIO 6 — URL fournie par l'utilisateur

Vulnérabilité

SSRF — Server-Side Request Forgery

Justification

Le serveur accède à une URL contrôlée par l'utilisateur sans vérifier correctement l'hôte ou l'adresse IP de destination.

Exploitation / PoC — éléments attendus

- Identifier le paramètre acceptant une URL.
- Fournir une URL de test contrôlée.
- Vérifier que le serveur effectue réellement la requête.
- Tester une destination interne autorisée dans l'environnement de laboratoire.
- Observer la réponse.
- Déterminer si l'application peut atteindre des ressources normalement inaccessibles à l'utilisateur.

Impact potentiel

- Accès à des services internes.
- Divulcation d'informations internes.
- Utilisation du serveur comme intermédiaire vers d'autres ressources.

Prévention

Techniques :

- Valider strictement les URL/destinations autorisées.
- Bloquer l'accès aux adresses et services internes non nécessaires.

Organisationnelles :

- Définir les ressources auxquelles l'application peut accéder.
- Appliquer le principe du moindre privilège réseau.

Tests automatisés :

- Tester les URL externes, locales et destinations interdites.

SCÉNARIO 7 — Désérialisation d'un objet fourni par le client

Vulnérabilité

Désérialisation non sécurisée

Justification

L'application désérialise directement des données contrôlées par le client sans validation ni restriction.

Exploitation / PoC — éléments attendus

- Identifier les données sérialisées envoyées au serveur.
- Modifier de façon contrôlée certaines valeurs.
- Envoyer l'objet modifié.
- Observer comment l'application le désérialise.
- Vérifier si des propriétés ou comportements non prévus sont acceptés.
- Conclure sur l'absence de contrôle suffisant.

Impact potentiel

Selon la technologie et le contexte :

- modification du comportement de l'application ;
- manipulation de données ;
- dans certains cas, exécution de code.

Prévention

Techniques :

- Éviter de désérialiser des objets non fiables.
- Utiliser des formats simples et valider strictement les données attendues.

Organisationnelles :

- Encadrer l'utilisation des mécanismes de sérialisation.
- Revue de code des fonctions sensibles.

Tests automatisés :

- Tester les objets modifiés, inattendus ou malformés.

SCÉNARIO 8 — Upload d'images + inclusion locale vulnérable

Vulnérabilité

Upload de fichiers non sécurisés associé à une LFI

Justification

Les fichiers uploadés sont placés dans un répertoire publiquement accessible et une autre fonction permet une inclusion locale vulnérable. La combinaison augmente fortement le risque.

Exploitation / PoC — éléments attendus

- Identifier la fonction d'upload.
- Envoyer un fichier de test non dangereux.
- Vérifier son emplacement.
- Vérifier s'il est directement accessible.
- Identifier la fonction d'inclusion locale.
- Vérifier en laboratoire si le fichier uploadé peut être référencé par cette fonction.

Impact potentiel

- Dépôt de fichiers non autorisés.
- Accès à des fichiers locaux.
- Contournement de certaines protections.
- Selon la configuration, impact plus important sur le serveur.

Prévention

Techniques :

- Vérifier type, extension, taille et contenu des fichiers.
- Stocker les uploads hors d'un répertoire directement exécutable/public.
- Corriger la vulnérabilité LFI.

Organisationnelles :

- Définir une politique d'upload.
- Réaliser des tests de sécurité sur les fonctions de fichiers.

Tests automatisés :

- Tester différents types/extensions et chemins de fichiers.

SCÉNARIO 9 — Contenu utilisateur interprété par le moteur de templates

Vulnérabilité

SSTI — Server-Side Template Injection

Justification

Le contenu contrôlé par l'utilisateur est interprété par le moteur de Template au lieu d'être traité uniquement comme une donnée.

Exploitation / PoC — éléments attendus

- Identifier le champ utilisé dans le Template.
- Envoyer une expression de test non dangereuse.
- Observer la réponse.
- Vérifier si l'expression est affichée comme texte ou évaluée.
- Si elle est évaluée, le moteur interprète le contenu utilisateur.
- La SSTI est alors mise en évidence.

Impact potentiel

- Accès à des informations internes.
- Manipulation du rendu.
- Selon le moteur et sa configuration, conséquences plus importantes côté serveur.

Prévention

Techniques :

- Ne pas interpréter directement les entrées utilisateur comme templates.
- Utiliser les mécanismes sûrs du moteur de templates et valider les données.

Organisationnelles :

- Respecter les pratiques de développement sécurisé du framework.
- Réaliser des revues de code.

Tests automatisés :

- Tester les entrées susceptibles d'être interprétées par le moteur de templates.

SCÉNARIO 10 — Dépendances externes non verrouillées

Vulnérabilité

Risque lié à la chaîne d'approvisionnement logicielle / dépendances

Justification

Le pipeline CI télécharge automatiquement des dépendances externes non verrouillées. Une version ou un paquet compromis peut donc être intégré automatiquement à l'application.

Exploitation / PoC — éléments attendus

- Identifier les dépendances récupérées pendant la build.
- Vérifier si leurs versions sont fixées.
- Vérifier leur origine.
- Observer le comportement du pipeline lors d'une nouvelle build.
- Identifier si une nouvelle version peut être récupérée automatiquement.
- Conclure sur le risque d'intégration d'une dépendance non maîtrisée.

Impact potentiel

- Introduction de code malveillant ou vulnérable.
- Compromission de l'application produite.
- Propagation du problème aux systèmes utilisant la build.

Prévention

Techniques :

- Verrouiller les versions des dépendances.
- Vérifier et analyser les dépendances avant intégration.

Organisationnelles :

- Utiliser des sources de confiance.
- Mettre en place une procédure de gestion et de validation des dépendances.

Tests automatisés :

- Scanner automatiquement les dépendances et signaler les composants vulnérables ou inattendus.

Synthèse de correction

Scénario	Réponse principale attendue
1	XSS
2	Injection SQL
3	Broken Access Control / IDOR
4	CSRF
5	Attaque automatisée contre l'authentification / Brute Force
6	SSRF
7	Désérialisation non sécurisée

8	Upload non sécurisé + LFI
9	SSTI
10	Supply Chain / dépendances compromises

Critères généraux de correction

Pour chaque scénario, vérifier que le stagiaire :

1. **Identifie correctement la vulnérabilité ;**
2. **Justifie à partir du scénario ;**
3. **Décrit une démarche de test cohérente ;**
4. **Identifie un impact réaliste ;**
5. **Propose au moins 2 mesures techniques ;**
6. **Propose au moins 2 mesures organisationnelles ;**
7. **Propose un test pouvant être automatisé ;**
8. **Justifie son niveau de risque ;**
9. **Donne 2 recommandations prioritaires cohérentes.**

Pour le **score de risque**, plusieurs réponses peuvent être acceptées si le stagiaire justifie correctement son évaluation.